

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

“Binary Tree dan Graf”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 10 Juni 2025

M. Fedora Argadyaksa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Pengertian Algoritma Binary Tree	3
2.2 Pengertian Algoritma Graph	3
2.2.1 Pengertian Algoritma BFS	3
2.2.2 Pengertian Algoritma DFS	3
2.3 Implementasi Stack pada Java.....	4
2.3.1 Class BTree (BTree_2511533016).....	4
2.3.2 Class Node (Node_2511533016)	6
2.3.3 Class Btree Driver (BTreeDriver_2511533016).....	9
2.3.4 Class Bubble Sort GUI (BubbleSortGUI_2511533016)	11
BAB III PENUTUP	15
3.1 Kesimpulan.....	15
3.2 Saran.....	15
DAFTAR PUSTAKA.....	16
LAPORAN TUGAS PRAKTIKUM	17
1. Class Lagu (Lagu_2511533016)	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu struktur data dan algoritma, pemahaman tentang representasi dan pengolahan data menjadi hal yang sangat penting untuk membangun sistem yang efisien. Salah satu struktur data yang sering digunakan adalah Binary Tree, yang memungkinkan penyimpanan data secara hierarkis untuk berbagai aplikasi seperti pencarian dan pengurutan data.

Selain itu, terdapat Graph yang digunakan untuk merepresentasikan hubungan yang kompleks seperti jaringan sosial, peta, dan sistem transportasi. Untuk melakukan penelusuran atau traversal pada struktur data tersebut, digunakan algoritma seperti Breadth First Search (BFS) dan Depth First Search (DFS). Kedua algoritma ini sangat penting dalam pemecahan masalah yang berkaitan dengan pencarian jalur, analisis jaringan, dan optimasi data.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 9 tentang materi diatas adalah sebagai berikut:

1. Memahami konsep dasar Binary Tree dan Graph sebagai struktur data non-linear.
2. Mampu menjelaskan perbedaan dan karakteristik masing-masing struktur data.
3. Mempelajari cara kerja algoritma BFS dan DFS dalam melakukan traversal data.
4. Mampu mengimplementasikan BFS dan DFS dalam penyelesaian masalah sederhana.
5. Mengembangkan kemampuan berpikir logis dan sistematis dalam pemrograman.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 9 tentang materi diatas adalah sebagai berikut:

1. Memudahkan dalam memahami struktur data kompleks yang digunakan dalam dunia nyata.
2. Meningkatkan kemampuan dalam merancang algoritma pencarian dan penelusuran data.
3. Membantu dalam menyelesaikan berbagai permasalahan seperti pencarian jalur terpendek, analisis jaringan, dan pemetaan data.
4. Menjadi dasar penting dalam pengembangan aplikasi seperti AI, game, dan sistem navigasi.
5. Melatih kemampuan analisis dan problem solving dalam bidang informatika.

BAB II

PEMBAHASAN

2.1 Pengertian Algoritma Binary Tree

Algoritma Binary Tree adalah metode pengolahan data pada struktur pohon biner, yaitu struktur data yang setiap nodenya memiliki maksimal dua anak (left child dan right child). Algoritma pada binary tree digunakan untuk melakukan operasi seperti penelusuran (traversal), pencarian, penyisipan, dan penghapusan data secara hierarkis.

2.2 Pengertian Algoritma Graph

Algoritma Graph adalah kumpulan langkah untuk mengolah struktur data graph yang terdiri dari simpul (vertex) dan sisi (edge). Algoritma ini digunakan untuk menyelesaikan masalah yang melibatkan hubungan antar objek, seperti pencarian jalur, konektivitas, dan jaringan. Contoh algoritma pada graph adalah BFS dan DFS.

2.2.1 Pengertian Algoritma BFS

Breadth First Search (BFS) adalah algoritma yang digunakan untuk menelusuri node pada graph atau tree secara bertingkat (level by level). BFS memulai dari node awal, kemudian mengunjungi semua tetangga terdekat terlebih dahulu sebelum melanjutkan ke level berikutnya. BFS biasanya menggunakan struktur data queue (antrian).

2.2.2 Pengertian Algoritma DFS

Depth First Search (DFS) adalah algoritma yang digunakan untuk menelusuri node dengan cara mengeksplorasi satu jalur sedalam mungkin terlebih dahulu, kemudian kembali (backtracking) untuk mengecek jalur lainnya. DFS biasanya menggunakan stack atau rekursi.

2.3 Implementasi Stack pada Java

Pada praktikum pekan 9 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat di bawah ini kode program yang dituliskan pada praktikum pekan 9.

2.3.1 Class BTree (BTree_2511533016)

```

1. package Pekan9_2511533016;
2.
3. public class BTree_2511533016 {
4.     private Node_2511533016 root_3016;
5.     private Node_2511533016 currentNode_3016;
6.     public BTree_2511533016() {
7.         root_3016 = null;
8.     }
9.     public boolean search_2511533016(int data_3016) {
10.         return search_2511533016(root_3016, data_3016);
11.     }
12.     private boolean search_2511533016(Node_2511533016
node_3016, int data_3016) {
13.         if (node_3016.getData_2511533016() == data_3016)
14.             return true;
15.         if (node_3016.getLeft_2511533016() != null)
16.             if
17.                 (search_2511533016(node_3016.getLeft_2511533016(), data_3016))
18.                     return true;
19.             if (node_3016.getRight_2511533016() != null)
20.                 if (search_2511533016(node_3016.getRight_2511533016(),
data_3016))
21.                     return true;
22.             return false;
23.         }
24.     public void printInorder_2511533016() {
25.         root_3016.printInorder_2511533016(root_3016);
26.     }
27.     public void printPreOrder_2511533016() {
28.         root_3016.printPreorder_2511533016(root_3016);
29.     }
30.     public void printPorstOrder_2511533016() {
31.         root_3016.printPostorder_2511533016(root_3016);
32.     }
33.     public Node_2511533016 getRoot_2511533016() {
34.         return root_3016;
35.     }
36.     public boolean isEmpty_2511533016() {
37.         return root_3016 == null;
38.     }
39. }

```

```

40.     public int countNodes_2511533016() {
41.         return countNodes_2511533016(root_3016);
42.     }
43.
44.     private int countNodes_2511533016(Node_2511533016
node_3016) {
45.         int count_3016 = 1;
46.         if(node_3016 == null) {
47.             return 0;
48.         } else {
49.             count_3016 +=
countNodes_2511533016(node_3016.getLeft_2511533016());
50.             count_3016 +=
countNodes_2511533016(node_3016.getRight_2511533016());
51.             return count_3016;
52.         }
53.     }
54.
55.     public void print_3016() {
56.         root_3016.print_3016();
57.     }
58.
59.     public Node_2511533016 getCurrent_3016() {
60.         return currentNode_3016;
61.     }
62.
63.     public void setCurrent_3016(Node_2511533016 node_3016) {
64.         this.currentNode_3016 = node_3016;
65.     }
66.
67.     public void setRoot_3016(Node_2511533016 root_3016) {
68.         this.root_3016 = root_3016;
69.     }
70.
71. }

```

Gambar 2.2.1

Program di atas merupakan implementasi struktur data **Binary Tree** dalam bahasa Java yang dibungkus dalam kelas BTree_2511533016. Kelas ini memiliki atribut utama root_3016 yang berfungsi sebagai akar (root) dari pohon, serta currentNode_3016 yang digunakan untuk menyimpan node yang sedang aktif atau sedang diproses.

Di dalam kelas ini terdapat metode search_2511533016(int data_3016) yang digunakan untuk mencari suatu nilai dalam binary tree. Proses pencarian dilakukan secara rekursif melalui metode search_2511533016(Node_2511533016 node_3016, int data_3016) dengan cara mengecek node saat ini, kemudian jika tidak ditemukan akan dilanjutkan ke subtree kiri dan kanan hingga data ditemukan

atau seluruh node selesai diperiksa.

Program ini juga menyediakan beberapa metode traversal, yaitu `printInorder_2511533016()`, `printPreOrder_2511533016()`, dan `printPostOrder_2511533016()` yang masing-masing digunakan untuk menampilkan isi tree dengan urutan inorder, preorder, dan postorder. Semua metode ini memanfaatkan fungsi yang ada pada kelas `Node_2511533016`.

Selain itu, terdapat metode `countNodes_2511533016()` yang berfungsi untuk menghitung jumlah seluruh node dalam binary tree secara rekursif dengan menjumlahkan node pada subtree kiri dan kanan. Metode `isEmpty_2511533016()` digunakan untuk mengecek apakah tree masih kosong atau belum memiliki root. Program ini juga menyediakan getter dan setter seperti `getRoot_2511533016()`, `setRoot_3016(Node_2511533016 root_3016)`, `getCurrent_3016()`, dan `setCurrent_3016(Node_2511533016 node_3016)` yang berfungsi untuk mengakses serta mengubah nilai root dan node yang sedang aktif dalam struktur tree.

Kesimpulan : Program di atas menunjukkan implementasi struktur data Binary Tree (pohon biner) yang dilengkapi dengan berbagai operasi dasar seperti pencarian data (search), traversal (inorder, preorder, postorder), perhitungan jumlah node, serta pengecekan kondisi tree. Dengan adanya implementasi ini, program dapat digunakan untuk mengelola data secara hierarkis dan mendukung proses penelusuran data secara rekursif dengan efisien.

2.3.2 Class Node (Node_2511533016)

```
package Pekan9_2511533016;

public class Node_2511533016 {
    int data_3016;
    Node_2511533016 left_3016;
    Node_2511533016 right_3016;
    public Node_2511533016(int data_3016) {
        this.data_3016 = data_3016;
        left_3016 = null;
        right_3016 = null;
    }
    public void setLeft_2511533016(Node_2511533016 node_3016) {
```

```

        if(left_3016 == null)
            left_3016 = node_3016;
    }
    public void setRight_2511533016(Node_2511533016 node_3016) {
        if(right_3016 == null)
            right_3016 = node_3016;
    }
    public Node_2511533016 getLeft_2511533016() {
        return left_3016;
    }
    public Node_2511533016 getRight_2511533016() {
        return right_3016;
    }
    public int getData_2511533016() {
        return data_3016;
    }
    public void setData_2511533016(int data_3016) {
        this.data_3016 = data_3016;
    }

    void printPreorder_2511533016(Node_2511533016 node_3016) {
        if(node_3016 == null)
            return;
        System.out.print(node_3016.data_3016 + " ");
        printPreorder_2511533016(node_3016.left_3016);
        printPreorder_2511533016(node_3016.right_3016);
    }
    void printPostorder_2511533016(Node_2511533016 node_3016) {
        if(node_3016 == null)
            return;
        printPostorder_2511533016(node_3016.left_3016);
        printPostorder_2511533016(node_3016.right_3016);
        System.out.print(node_3016.data_3016 + " ");
    }
    void printInorder_2511533016(Node_2511533016 node_3016) {
        if(node_3016 == null)
            return;
        printInorder_2511533016(node_3016.left_3016);
        System.out.print(node_3016.data_3016 + " ");
        printInorder_2511533016(node_3016.right_3016);
    }
    public String print_3016() {
        return this.print_3016("",true,"");
    }
    public String print_3016(String prefix_3016, boolean isTail_3016,
String sb_3016) {
        if(right_3016 != null) {
            right_3016.print_3016(prefix_3016 + (isTail_3016 ?
"| " : " "), false, sb_3016);
        }
        System.out.println( prefix_3016 + (isTail_3016 ? "\\-- " :
" /-- ") + data_3016);
        if(left_3016 != null) {
            left_3016.print_3016(prefix_3016 + (isTail_3016 ? "
: "| " : " "), true, sb_3016);
        }
        return sb_3016;
    }

```

```

    }
}

```

Gambar 2.2.2

Program di atas merupakan implementasi kelas Node pada struktur data Binary Tree dalam bahasa Java. Kelas Node_2511533016 berfungsi sebagai representasi satu simpul (node) yang menyimpan data bertipe integer (data_3016) serta memiliki dua referensi anak, yaitu left_3016 untuk anak kiri dan right_3016 untuk anak kanan. Pada konstruktor, setiap node akan diinisialisasi dengan nilai data tertentu dan secara default kedua child diset ke null.

Kelas ini juga menyediakan metode setLeft_2511533016() dan setRight_2511533016() untuk menghubungkan node dengan anak kiri dan kanan, namun hanya akan mengisi jika posisi tersebut masih kosong. Selain itu terdapat metode getter dan setter seperti getLeft_2511533016(), getRight_2511533016(), getData_2511533016(), dan setData_2511533016() yang digunakan untuk mengakses serta memodifikasi nilai dan hubungan antar node.

Program ini juga memiliki metode traversal, yaitu printPreorder_2511533016(), printInorder_2511533016(), dan printPostorder_2511533016() yang digunakan untuk menelusuri dan mencetak isi tree dengan urutan berbeda. Ketiga metode ini bekerja secara rekursif dengan mengunjungi node saat ini, kemudian subtree kiri dan kanan sesuai dengan jenis traversal.

Selain itu, terdapat metode print_3016() dan print_3016(String prefix_3016, boolean isTail_3016, String sb_3016) yang digunakan untuk menampilkan struktur tree dalam bentuk visual (seperti pohon) di output console. Metode ini membantu menggambarkan hubungan antar node secara hierarkis sehingga lebih mudah dipahami.

Kesimpulan: Program di atas merupakan implementasi dari struktur data Binary Tree pada level Node (simpul) yang berfungsi sebagai elemen dasar penyusun pohon biner. Setiap node menyimpan data serta memiliki hubungan ke node kiri dan kanan, serta dilengkapi dengan berbagai metode traversal (preorder, inorder, postorder) dan metode visualisasi tree. Dengan demikian, program ini

menunjukkan bagaimana setiap node dalam binary tree bekerja sebagai unit dasar yang membentuk keseluruhan struktur pohon dan mendukung operasi penelusuran data secara rekursif.

2.3.3 Class Btree Driver (BTreeDriver_2511533016)

```

1. package Pekan9_2511533016;
2.
3. public class BtreeDriver_2511533016 {
4.
5.     public static void main(String[] args) {
6.         //Membuat Pohon
7.         BTree_2511533016 tree_3016 = new BTree_2511533016();
8.         System.out.print("Jumlah Simpul awal pohon : ");
9.
10.        System.out.println(tree_3016.countNodes_2511533016());
11.        //menambahkan simpul data 1
12.        Node_2511533016 root_3016 = new Node_2511533016(1);
13.        //menjadikan simpul 1 sebagai root
14.        tree_3016.setRoot_3016(root_3016);
15.        System.out.println("Jumlah simpul jika hanya ada
    root");
16.        System.out.println(tree_3016.countNodes_2511533016());
17.        Node_2511533016 node2_3016 = new Node_2511533016(2);
18.        Node_2511533016 node3_3016 = new Node_2511533016(3);
19.        Node_2511533016 node4_3016 = new Node_2511533016(4);
20.        Node_2511533016 node5_3016 = new Node_2511533016(5);
21.        Node_2511533016 node6_3016 = new Node_2511533016(6);
22.        Node_2511533016 node7_3016 = new Node_2511533016(7);
23.        Node_2511533016 node8_3016 = new Node_2511533016(8);
24.        Node_2511533016 node9_3016 = new Node_2511533016(9);
25.        root_3016.setLeft_2511533016(node2_3016);
26.        node2_3016.setLeft_2511533016(node4_3016);
27.        node2_3016.setRight_2511533016(node5_3016);
28.        node4_3016.setRight_2511533016(node8_3016);
29.        root_3016.setRight_2511533016(node3_3016);
30.        node3_3016.setLeft_2511533016(node6_3016);
31.        node3_3016.setRight_2511533016(node7_3016);
32.        node6_3016.setLeft_2511533016(node9_3016);
33.        //Set root
34.        tree_3016.setCurrent_3016(tree_3016.getRoot_2511533016());
35.        System.out.println("Menampilkan simpul terakhir :
    ");
36.        System.out.println(tree_3016.getCurrent_3016().getData_2511
    533016());
37.        System.out.println("Jumlah simpul; setelah simpul 7
    ditambahkan ");
38.        System.out.println(tree_3016.countNodes_2511533016());
39.        System.out.println("InOrder : ");
40.        tree_3016.printInorder_2511533016();

```

```

40.         System.out.println("\nPreorder : ");
41.         tree_3016.printPorstOrder_2511533016();
42.         System.out.println("\nDmenampilkan simpul dalam
    bentuk pohon");
43.         tree_3016.print_3016();
44.     }
45.
46. }

```

```

Jumlah Simpul awal pohon : 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir :
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder :
4 8 2 5 1 9 6 3 7
Preorder :
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|
|-- 7
|   |-- 3
|       |-- 6
|           |-- 9
|-- 1
    |-- 5
        |-- 2
            |-- 8
                |-- 4

```

Gambar 2.2.3

Program di atas merupakan kelas driver (main program) yang digunakan untuk menguji implementasi struktur data Binary Tree yang telah dibuat pada kelas BTree_2511533016 dan Node_2511533016. Pada awal program, dibuat objek tree bernama tree_3016, kemudian dilakukan pengecekan jumlah simpul (node) menggunakan metode countNodes_2511533016() saat tree masih kosong, sehingga hasilnya menunjukkan bahwa belum ada node yang terbentuk.

Selanjutnya, dibuat node root dengan nilai 1 dan ditetapkan sebagai root dari binary tree menggunakan metode setRoot_3016(). Setelah itu, kembali dilakukan pengecekan jumlah node yang kini hanya berisi root saja. Program kemudian membuat beberapa node tambahan (node 2 hingga node 9) yang akan disusun membentuk struktur binary tree.

Node-node tersebut dihubungkan menggunakan metode setLeft_2511533016() dan setRight_2511533016() untuk membentuk struktur pohon sesuai hierarki, mulai dari root hingga cabang-cabangnya. Setelah struktur tree terbentuk, program menyimpan posisi root ke dalam variabel current node dan menampilkannya sebagai node terakhir yang sedang diakses.

Program juga menampilkan jumlah node setelah seluruh node ditambahkan, kemudian melakukan proses traversal menggunakan metode Inorder dan Postorder (meskipun pada output terdapat penamaan yang kurang tepat pada

bagian pemanggilan preorder/postorder). Selain itu, program menampilkan struktur binary tree dalam bentuk visual menggunakan metode print_3016(), sehingga hubungan antar node dapat terlihat lebih jelas dalam bentuk pohon.

Kesimpulan : Program di atas merupakan implementasi driver Binary Tree yang berfungsi untuk membentuk, menghubungkan, dan menguji operasi pada struktur data binary tree. Program ini menunjukkan proses pembuatan node, penyusunan tree secara manual, perhitungan jumlah node, serta penerapan traversal dan visualisasi tree. Dengan demikian, program ini berperan sebagai uji coba (testing) implementasi Binary Tree lengkap dengan operasi dasar seperti insert, traversal, dan representasi struktur pohon.

2.3.4 Class Bubble Sort GUI (BubbleSortGUI_2511533016)

```

1. package Pekan9_2511533016;
2.
3. import java.util.*;
4. public class GraphTraversal_2511533016 {
5.     private Map<String, List<String>> graph = new HashMap<>();
6.
7.     // Menambahkan edge (graf tak berarah)
8.     public void addEdge_2511533016(String node1_3016, String
node2_3016) {
9.         graph.putIfAbsent(node1_3016, new ArrayList<>());
10.        graph.putIfAbsent(node2_3016, new ArrayList<>());
11.        graph.get(node1_3016).add(node2_3016);
12.        graph.get(node2_3016).add(node1_3016);
13.    }
14.    // Menampilkan graf awal
15.    public void printGraph_2511533016() {
16.        System.out.println("Graf Awal (Adjacency List) : ");
17.        for(String node_3016 : graph.keySet()) {
18.            System.out.print(node_3016 + " -> ");
19.            List<String> neighbors_3016 =
graph.get(node_3016);
20.            System.out.println(String.join(", ",
neighbors_3016));
21.        }
22.        System.out.println();
23.    }
24.
25.    // DFS rekursif
26.    public void dfs_2511533016(String start_3016) {
27.        Set<String> visited_3016 = new HashSet<>();
28.        System.out.println("Penelusuran DFS : ");
29.        dfsHelper_2511533016(start_3016, visited_3016);
30.        System.out.println();

```

```

31.     }
32.     private void dfsHelper_2511533016(String current_3016,
Set<String> visited_3016) {
33.         if(visited_3016.contains(current_3016)) return;
34.         visited_3016.add(current_3016);
35.         System.out.print(current_3016 + " ");
36.         for (String neighbor_3016 :
graph.getDefault(current_3016, new ArrayList<>())) {
37.             dfsHelper_2511533016(neighbor_3016,
visited_3016);
38.         }
39.     }
40.     // BFS iteratif
41.     public void bfs_2511533016(String start_3016) {
42.         Set<String> visited_3016 = new HashSet<>();
43.         Queue<String> queue_3016 = new LinkedList<>();
44.         queue_3016.add(start_3016);
45.         visited_3016.add(start_3016);
46.         System.out.println("Penelusuran BFS : ");
47.         while(!queue_3016.isEmpty()) {
48.             String current_3016 = queue_3016.poll();
49.             System.out.print(current_3016 + " ");
50.             for(String neighbor :
graph.getDefault(current_3016, new ArrayList<>()))
51.                 if(!visited_3016.contains(neighbor)) {
52.                     queue_3016.add(neighbor);
53.                     visited_3016.add(neighbor);
54.                 }
55.             }
56.             System.out.println();
57.         }
58.         //Main
59.         public static void main(String[] args) {
60.             GraphTraversal_2511533016 graph_3016 = new
GraphTraversal_2511533016();
61.
62.             // Contoh graf : A-B, A-C, B-D, B-E
63.             graph_3016.addEdge_2511533016("A", "B");
64.             graph_3016.addEdge_2511533016("A", "C");
65.             graph_3016.addEdge_2511533016("B", "D");
66.             graph_3016.addEdge_2511533016("B", "E");
67.             // Cetak graf awal
68.             System.out.println("Graf Awal adalah : ");
69.             graph_3016.printGraph_2511533016();
70.             // Lakukan penelusuran
71.             graph_3016.dfs_2511533016("A");
72.             graph_3016.bfs_2511533016("A");
73.         }
74.
75.
76. }

```

```

Graf Awal adalah :
Graf Awal (Adjacency List) :
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS :
A B D E C
Penelusuran BFS :
A B C D E

```

Gambar 2.2.4

Program di atas merupakan implementasi struktur data Graph beserta algoritma penelusurannya, yaitu Depth First Search (DFS) dan Breadth First Search (BFS) menggunakan bahasa pemrograman Java. Graph direpresentasikan menggunakan adjacency list, yaitu struktur data `Map<String, List<String>>` yang menyimpan setiap node beserta daftar tetangganya.

Pada bagian awal, metode `addEdge_2511533016()` digunakan untuk menambahkan hubungan antar node (edge) pada graph. Graph yang digunakan bersifat tidak berarah (undirected graph), sehingga setiap hubungan akan ditambahkan dua arah, yaitu dari node pertama ke node kedua dan sebaliknya. Selanjutnya, metode `printGraph_2511533016()` digunakan untuk menampilkan isi graph dalam bentuk adjacency list agar hubungan antar node dapat terlihat dengan jelas.

Program ini juga mengimplementasikan algoritma DFS (Depth First Search) secara rekursif melalui metode `dfs_2511533016()` dan `dfsHelper_2511533016()`. Algoritma ini bekerja dengan cara menelusuri node sedalam mungkin terlebih dahulu sebelum kembali ke cabang lainnya, serta menggunakan struktur data Set untuk menyimpan node yang sudah dikunjungi agar tidak terjadi perulangan.

Selain DFS, program ini juga mengimplementasikan BFS (Breadth First Search) melalui metode `bfs_2511533016()` yang menggunakan struktur data queue (antrian). BFS menelusuri graph secara bertahap berdasarkan level, di mana node yang bertetangga akan dikunjungi terlebih dahulu sebelum berpindah ke level berikutnya.

Pada bagian main, program membuat contoh graph sederhana dengan node A, B, C, D, dan E, kemudian menambahkan hubungan antar node menggunakan

metode `addEdge_2511533016()`. Setelah graph terbentuk, program menampilkan representasi graph awal, lalu melakukan penelusuran menggunakan DFS dan BFS mulai dari node A.

Kesimpulan: Program di atas merupakan implementasi struktur data Graph menggunakan adjacency list yang dilengkapi dengan algoritma traversal DFS dan BFS. Program ini menunjukkan bagaimana hubungan antar node dapat direpresentasikan serta bagaimana proses penelusuran data dilakukan dengan dua metode berbeda, yaitu penelusuran mendalam (DFS) dan penelusuran melebar (BFS). Dengan demikian, program ini berfungsi sebagai simulasi dasar pengolahan graph dalam pemrograman dan penerapan algoritma pencarian pada struktur data non-linear.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan pembahasan dan implementasi program, dapat disimpulkan bahwa Binary Tree dan Graph merupakan struktur data non-linear yang digunakan untuk merepresentasikan data yang memiliki hubungan tertentu. Binary Tree digunakan untuk menyimpan data secara hierarkis dengan setiap node memiliki maksimal dua anak, sedangkan Graph digunakan untuk merepresentasikan hubungan yang lebih kompleks antara banyak node tanpa batasan jumlah hubungan.

Selain itu, algoritma traversal seperti Depth First Search (DFS) dan Breadth First Search (BFS) sangat penting dalam proses penelusuran data pada struktur tree maupun graph. DFS bekerja dengan cara menjelajah node secara mendalam terlebih dahulu sebelum kembali (backtracking), sedangkan BFS menjelajah node secara bertingkat (level per level) menggunakan antrian. Dari implementasi program yang dibuat, dapat dipahami bahwa penggunaan rekursi, stack, dan queue sangat berperan dalam proses traversal. Secara keseluruhan, materi ini memberikan pemahaman dasar yang kuat mengenai bagaimana data terstruktur dan bagaimana algoritma digunakan untuk mengakses serta mengolah data tersebut secara efisien.

3.2 Saran

Dalam mempelajari materi Binary Tree, Graph, BFS, dan DFS, disarankan untuk tidak hanya memahami teori, tetapi juga melakukan praktik langsung melalui pemrograman agar lebih mudah memahami alur kerja algoritma. Pemahaman konsep rekursi juga sangat penting karena banyak digunakan dalam implementasi tree dan DFS.

Selain itu, sebaiknya mencoba berbagai variasi bentuk tree dan graph agar lebih memahami perbedaan hasil traversal. Penggunaan alat bantu visualisasi juga dapat membantu dalam memahami proses penelusuran node secara lebih jelas.

DAFTAR PUSTAKA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013).
Data Structures and Algorithms in Java. Wiley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).
Introduction to Algorithms (3rd ed.). MIT Press.

LAPORAN TUGAS

PRAKTIKUM

SOAL :

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph

1. Class Lagu (Lagu_2511533016)

```
1. package Pekan9_2511533016;
2.
3. import javax.swing.*;
4. import javax.swing.border.*;
5. import java.awt.*;
6. import java.util.Arrays;
7. import java.util.ArrayList;
8. import java.util.LinkedList;
9. import java.util.List;
10. import java.util.Queue;
11. import java.util.Stack;
12.
13. public class PetaKampus_2511533016 extends JFrame {
14.
15.     // ===== 10 NODE, 15 EDGE =====
16.     private final String[] nodes_3016 = {
17.         "Gerbang",    // 0
18.         "Rektorat",   // 1
19.         "Perpus",     // 2
20.         "FTI",        // 3
21.         "FKIP",       // 4
22.         "Lab Komp",   // 5
23.         "Lab AI",     // 6
24.         "Masjid",     // 7
25.         "Kantin",     // 8
26.         "PKM"         // 9
27.     };
28.
29.     // 15 edge
30.     private final int[][] edges_3016 = {
31.         {0, 1}, // Gerbang - Rektorat
32.         {0, 8}, // Gerbang - Kantin
33.         {1, 2}, // Rektorat - Perpus
34.         {1, 4}, // Rektorat - FKIP
35.         {1, 9}, // Rektorat - PKM
36.         {2, 3}, // Perpus - FTI
37.         {2, 9}, // Perpus - PKM
```

```

38.         {3, 5}, // FTI      - Lab Komp
39.         {3, 6}, // FTI      - Lab AI
40.         {4, 7}, // FKIP      - Masjid
41.         {4, 9}, // FKIP      - PKM
42.         {5, 6}, // Lab Komp - Lab AI
43.         {6, 7}, // Lab AI     - Masjid
44.         {7, 9}, // Masjid    - PKM
45.         {8, 9}, // Kantin    - PKM
46.     };
47.
48.     private JComboBox<String> cbStart_3016, cbGoal_3016;
49.     private JTextArea taGraph_3016, taResult_3016;
50.
51.     public PetaKampus_2511533016() {
52.         setTitle("Peta Kampus - BFS & DFS | 2511533016");
53.         setDefaultCloseOperation(EXIT_ON_CLOSE);
54.         setLayout(new BorderLayout(4, 4));
55.
56.         buildTop_3016();
57.         buildGraph_3016();
58.         buildResult_3016();
59.
60.         setSize(750, 520);
61.         setLocationRelativeTo(null);
62.         setVisible(true);
63.     }
64.
65.     private void buildTop_3016() {
66.         JPanel north_3016 = new JPanel(new BorderLayout());
67.
68.         JLabel title_3016 = new JLabel("PENCARIAN JALUR
        MENGGUNAKAN BFS DAN DFS", SwingConstants.CENTER);
69.         title_3016.setFont(new Font("Dialog", Font.BOLD, 15));
70.         title_3016.setForeground(Color.WHITE);
71.         title_3016.setOpaque(true);
72.         title_3016.setBackground(new Color(50, 80, 130));
73.         title_3016.setBorder(BorderFactory.createEmptyBorder(8,
        8, 8, 8));
74.
75.         JPanel ctrl_3016 = new JPanel(new
        FlowLayout(FlowLayout.LEFT, 10, 6));
76.         ctrl_3016.setBackground(new Color(230, 230, 230));
77.
78.         cbStart_3016 = new JComboBox<>(nodes_3016);
79.         cbStart_3016.setSelectedIndex(0);
80.         cbGoal_3016 = new JComboBox<>(nodes_3016);
81.         cbGoal_3016.setSelectedIndex(6);
82.
83.         JButton btnBFS_3016 = buatTombol_3016("[ BFS ]", new
        Color(60, 160, 60));
84.         JButton btnDFS_3016 = buatTombol_3016("[ DFS ]", new
        Color(200, 130, 0));
85.         JButton btnReset_3016 = buatTombol_3016("[ RESET ]", new
        Color(190, 40, 40));
86.
87.         btnBFS_3016.addActionListener(e -> {
88.             resetGraph_2511533016();

```

```

89.         BFS_2511533016(cbStart_3016.getSelectedIndex(),
        cbGoal_3016.getSelectedIndex());
90.     });
91.     btnDFS_3016.addActionListener(e -> {
92.         resetGraph_2511533016();
93.         DFS_2511533016(cbStart_3016.getSelectedIndex(),
        cbGoal_3016.getSelectedIndex());
94.     });
95.     btnReset_3016.addActionListener(e ->
        resetGraph_2511533016());
96.
97.     ctrl_3016.add(new JLabel("Lokasi Awal :"));
98.     ctrl_3016.add(cbStart_3016);
99.     ctrl_3016.add(new JLabel("Lokasi Tujuan :"));
100.    ctrl_3016.add(cbGoal_3016);
101.    ctrl_3016.add(btnBFS_3016);
102.    ctrl_3016.add(btnDFS_3016);
103.    ctrl_3016.add(btnReset_3016);
104.
105.    north_3016.add(title_3016, BorderLayout.NORTH);
106.    north_3016.add(ctrl_3016, BorderLayout.SOUTH);
107.    add(north_3016, BorderLayout.NORTH);
108.    }
109.
110.    private JButton buatTombol_3016(String teks_3016, Color
        warna_3016) {
111.        JButton btn_3016 = new JButton(teks_3016);
112.        btn_3016.setBackground(warna_3016);
113.        btn_3016.setForeground(Color.WHITE);
114.        btn_3016.setFont(new Font("Dialog", Font.BOLD,
        13));
115.        btn_3016.setFocusPainted(false);
116.        return btn_3016;
117.    }
118.
119.    private void buildGraph_3016() {
120.        taGraph_3016 = new JTextArea();
121.        taGraph_3016.setFont(new Font("Monospaced",
        Font.PLAIN, 13));
122.        taGraph_3016.setEditable(false);
123.        taGraph_3016.setBackground(Color.WHITE);
124.        taGraph_3016.setText(displayGraph_2511533016());
125.
126.        JScrollPane sp_3016 = new
        JScrollPane(taGraph_3016);
127.        sp_3016.setBorder(BorderFactory.createTitledBorder(
        BorderFactory.createLineBorder(Color.GRAY),
128.        "VISUALISASI GRAPH", TitledBorder.LEFT,
        TitledBorder.TOP,
129.        new Font("Dialog", Font.BOLD, 12)));
130.        sp_3016.setPreferredSize(new Dimension(750, 240));
131.        add(sp_3016, BorderLayout.CENTER);
132.    }
133.
134.    private void buildResult_3016() {
135.        taResult_3016 = new JTextArea(6, 70);

```

```

137.         taResult_3016.setFont(new Font("Monospaced",
Font.BOLD, 13));
138.         taResult_3016.setEditable(false);
139.         taResult_3016.setBackground(Color.WHITE);
140.         taResult_3016.setText(
141.             "Hasil Pencarian :\n" +
142.             "Jalur :\n" +
143.             "Node Dikunjungi :\n" +
144.             "Jumlah Node Dikunjungi : 0"
145.         );
146.         JScrollPane sp_3016 = new
JScrollPane(taResult_3016);
147.         sp_3016.setBorder(BorderFactory.createLineBorder(Color.GRAY));
148.         add(sp_3016, BorderLayout.SOUTH);
149.     }
150.
151.     // ===== displayGraph =====
152.     public String displayGraph_2511533016() {
153.         return
154.             "\n" +
155.             "    Gerbang ----- Rektorat ----- Perpus
----- FTI\n" +
156.             "        |                /          \\          \\
/    \\ \\n" +
157.             "    Kantin          FKIP          PKM -----+
Lab Komp Lab AI\n" +
158.             "        |                |          / |
|\n" +
159.             "    +-----> PKM          Kantin
Masjid\n" +
160.             "                |
|\n" +
161.             "                Masjid <-----
-----PKM\n" +
162.             "\n" +
163.             "    Total Node: 10 | Total Edge: 15\n" +
164.             "    Peta: Jalur Antar Lokasi di Kampus\n";
165.     }
166.
167.     // ===== BFS =====
168.     public List<Integer> BFS_2511533016(int start_3016, int
goal_3016) {
169.         boolean[] visited_3016 = new
boolean[nodes_3016.length];
170.         int[] parent_3016 = new
int[nodes_3016.length];
171.         List<Integer> order_3016 = new ArrayList<>();
172.         Arrays.fill(parent_3016, -1);
173.
174.         Queue<Integer> queue_3016 = new LinkedList<>();
175.         queue_3016.add(start_3016);
176.         visited_3016[start_3016] = true;
177.
178.         while (!queue_3016.isEmpty()) {
179.             int curr_3016 = queue_3016.poll();
180.             order_3016.add(curr_3016);

```

```

181.         if (curr_3016 == goal_3016) break;
182.         for (int[] e_3016 : edges_3016) {
183.             int nb_3016 = tetangga_3016(e_3016,
curr_3016, visited_3016);
184.             if (nb_3016 != -1) {
185.                 visited_3016[nb_3016] = true;
186.                 parent_3016[nb_3016] = curr_3016;
187.                 queue_3016.add(nb_3016);
188.             }
189.         }
190.     }
191.     displayPath_2511533016(start_3016, goal_3016,
parent_3016, order_3016, "BFS");
192.     return bangunJalur_3016(goal_3016, parent_3016);
193. }
194.
195. // ===== DFS =====
196. public List<Integer> DFS_2511533016(int start_3016, int
goal_3016) {
197.     boolean[] visited_3016 = new
boolean[nodes_3016.length];
198.     int[] parent_3016 = new
int[nodes_3016.length];
199.     List<Integer> order_3016 = new ArrayList<>();
200.     Arrays.fill(parent_3016, -1);
201.
202.     Stack<Integer> stack_3016 = new Stack<>();
203.     stack_3016.push(start_3016);
204.
205.     while (!stack_3016.isEmpty()) {
206.         int curr_3016 = stack_3016.pop();
207.         if (visited_3016[curr_3016]) continue;
208.         visited_3016[curr_3016] = true;
209.         order_3016.add(curr_3016);
210.         if (curr_3016 == goal_3016) break;
211.         for (int[] e_3016 : edges_3016) {
212.             int nb_3016 = tetangga_3016(e_3016,
curr_3016, visited_3016);
213.             if (nb_3016 != -1) {
214.                 parent_3016[nb_3016] = curr_3016;
215.                 stack_3016.push(nb_3016);
216.             }
217.         }
218.     }
219.     displayPath_2511533016(start_3016, goal_3016,
parent_3016, order_3016, "DFS");
220.     return bangunJalur_3016(goal_3016, parent_3016);
221. }
222.
223. private int tetangga_3016(int[] edge_3016, int
curr_3016, boolean[] visited_3016) {
224.     if (edge_3016[0] == curr_3016 &&
!visited_3016[edge_3016[1]]) return edge_3016[1];
225.     if (edge_3016[1] == curr_3016 &&
!visited_3016[edge_3016[0]]) return edge_3016[0];
226.     return -1;
227. }

```



```

228.
229.         private List<Integer> bangunJalur_3016(int goal_3016,
230.             int[] parent_3016) {
231.             List<Integer> path_3016 = new ArrayList<>();
232.             for (int at_3016 = goal_3016; at_3016 != -1;
233.                 at_3016 = parent_3016[at_3016])
234.                 path_3016.add(0, at_3016);
235.             return path_3016;
236.         }
237.         // ===== displayPath =====
238.         public void displayPath_2511533016(int start_3016, int
239.             goal_3016, int[] parent_3016,
240.             List<Integer>
241.             order_3016, String algo_3016) {
242.             List<Integer> path_3016 =
243.                 bangunJalur_3016(goal_3016, parent_3016);
244.             boolean found_3016 = !path_3016.isEmpty() &&
245.                 path_3016.get(0) == start_3016;
246.             StringBuilder sb_3016 = new StringBuilder();
247.             sb_3016.append("Hasil Pencarian :
248. ").append(algo_3016).append("\n");
249.             sb_3016.append("Jalur          : ");
250.             if (found_3016) {
251.                 for (int i_3016 = 0; i_3016 < path_3016.size();
252.                     i_3016++) {
253.                     sb_3016.append(nodes_3016[path_3016.get(i_3016)]);
254.                     if (i_3016 < path_3016.size() - 1)
255.                         sb_3016.append(" -> ");
256.                     }
257.                 } else {
258.                     sb_3016.append("Tidak ditemukan");
259.                 }
260.             sb_3016.append("\nNode Dikunjungi : ");
261.             for (int i_3016 = 0; i_3016 < order_3016.size();
262.                 i_3016++) {
263.                 sb_3016.append(nodes_3016[order_3016.get(i_3016)]);
264.                 if (i_3016 < order_3016.size() - 1)
265.                     sb_3016.append(", ");
266.                 }
267.             sb_3016.append("\nJumlah Node Dikunjungi :
268. ").append(order_3016.size());
269.             taResult_3016.setText(sb_3016.toString());
270.         }
271.         // ===== resetGraph =====
272.         public void resetGraph_2511533016() {
273.             taGraph_3016.setText(displayGraph_2511533016());
274.             taResult_3016.setText(
275.                 "Hasil Pencarian :\n" +
276.                 "Jalur :\n" +

```

```

271.         "Node Dikunjungi :\n" +
272.         "Jumlah Node Dikunjungi : 0"
273.     );
274.     }
275.
276.     public static void main(String[] args) {
277.         SwingUtilities.invokeLater(PetaKampus_2511533016::new);
278.     }
279. }

```

Gambar 1

Program di atas merupakan aplikasi berbasis Java Swing yang mengimplementasikan representasi graf sederhana untuk memodelkan peta lingkungan kampus serta melakukan pencarian jalur menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS). Graf direpresentasikan menggunakan array `nodes_3016` sebagai simpul (node) yang berisi lokasi-lokasi di kampus seperti Gerbang, Rektorat, Perpustakaan, hingga PKM, serta array `edges_3016` yang menyimpan hubungan antar node dalam bentuk pasangan indeks yang menunjukkan keterhubungan antar lokasi. Program ini membangun antarmuka grafis (GUI) menggunakan `JFrame`, `JComboBox`, `JTextArea`, dan `JButton` untuk memungkinkan pengguna memilih titik awal dan tujuan, kemudian menjalankan pencarian jalur menggunakan BFS atau DFS.

Pada bagian algoritma, BFS diimplementasikan menggunakan struktur data `Queue` untuk menelusuri node secara melebar (level-order traversal), sedangkan DFS menggunakan `Stack` untuk menelusuri node secara mendalam sebelum kembali (backtracking). Kedua algoritma ini juga mencatat urutan node yang dikunjungi serta menyimpan parent node untuk membentuk kembali jalur dari tujuan ke awal. Fungsi `tetangga_3016` digunakan untuk mencari node tetangga yang belum dikunjungi berdasarkan daftar edge, sedangkan `bangunJalur_3016` bertugas merekonstruksi jalur hasil pencarian. Hasil pencarian kemudian ditampilkan pada `JTextArea`, termasuk jalur yang ditemukan, urutan node yang dikunjungi, dan jumlah node yang dieksplorasi. Selain itu, terdapat fitur reset untuk mengembalikan tampilan ke kondisi awal.

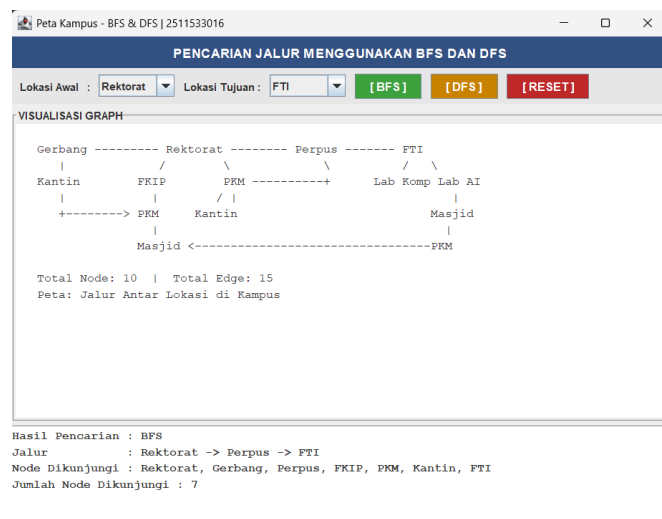
Kesimpulan : program ini menunjukkan implementasi nyata struktur data graf beserta penerapan algoritma BFS dan DFS dalam penyelesaian masalah pencarian

jalur (pathfinding). Aplikasi ini juga memperlihatkan bagaimana konsep teori graf dapat diintegrasikan dengan GUI untuk memberikan visualisasi dan interaksi yang lebih mudah dipahami. Program ini dapat digunakan sebagai media pembelajaran untuk memahami perbedaan cara kerja BFS dan DFS dalam penelusuran graf serta efektivitasnya dalam menemukan jalur antara dua titik.

OUTPUT :

CONTOH REKTORAT KE FTI

BFS :



DFS :

